

MSc Bioinformatics Coursework

BIO37427

Anneke Prins

Submitted on 2026-09-04

1. Simulation Exercise: Snakes and Ladders

Snakes and ladders is a board game that originated in ancient India. It is played on a gridded game board of 8x8, 10x10 or 12x12 squares. While the game is based on chance, it has its roots in morality lessons where ladders represent virtues and snakes represent vices. Mathematically speaking, it can be represented by a Markov chain (Sorkin, 2023) since the probability of moving to any square from a given square is fixed and independent of any previous game history. This assignment uses a board with 10x10 squares and 19 snakes and ladders (10 snakes and 9 ladders).

In this exercise I will first comment on existing code (Question 1).

I will then extend the code to do the following (Question 2):

- incorporate the effect of snakes and ladders
- simulate 1000 turns, plotting the position of each player at each turn (assuming reaching or passing square 100 = a win)
- compute the step at which a player exceeds square number 60 on the board

For Question 3 I will calculate for a single player:

- how many turns it takes to reach the 100th square
- repeat the simulation 1000 times
- plot the distribution of the number of turns taken to reach the 100th square
- explain my observations

References

Sorkin, G. B. (2023). Snakes and Ladders and Intransitivity, or what mathematicians do in their time off. *The Mathematical Intelligencer*, 45(4), 312-318.

Question 1

- Aim: comment on code

```

# set.seed creates reproducible random numbers (allows repeatable results)
set.seed(42)

# the maximum number of turns in the game (per player)
max_turns <- 20

# starting position is defined as 0 (outside of the board)
position <- 0

# the number of squares contained in the board; also the number for the highest
square
max_position <- 120

# the number of sides to the die; defines the probability of moving to any othe
r square (1/n_sides_die)
n_sides_die <- 4

# loop that simulates a game of snakes and ladders
for (turn in 1:max_turns) {

  # simulation of a die roll; sampling a single integer from the virtual die
  die_roll <- sample.int(n_sides_die, 1)

  # position on the board adjusted for the die roll
  position <- position + die_roll

  # defining the end of the game as the point where a player reaches the max p
osition or beyond
  if (position >= max_position) {
    break
  }
}

```

Question 2

- Aims:
 - Correct errors
 - Include snakes and ladders
 - Simulate 1000 turns
 - Plot position of player at each turn
 - Compute turn at which position > 60

```

# set seed for reproducibility
set.seed(42)

# max turns for simulated game is 1000
max_turns <- 1000

# start outside board; max position for 10x10 grid is 100; die has 6 sides and
is unbiased
# for this simulation we are assuming a 2-player game; the game ends when any g
iven player rolls a value that ends on square 100 or beyond
max_position <- 100
n_sides_die <- 6

# both players start at position 0
position_p1 <- 0
position_p2 <- 0

# define the start and end positions for the snakes and ladders; these are pair
ed by index
start_pos <- c(1, 4, 9, 16, 21, 28, 36, 48, 49, 51, 56, 62, 64, 71, 8
0, 87, 93, 95, 98)
end_pos <- c(38, 14, 31, 6, 42, 84, 44, 26, 11, 67, 53, 19, 60, 91, 1
00, 24, 73, 75, 78)

# initiate data frame for game
game_data = data.frame()

# initiate for loop to run game 1x
for (turn in 1:max_turns) {

  # player 1 plays
  die_roll_p1 <- sample.int(n_sides_die, 1)
  position_p1 <- position_p1 + die_roll_p1

  # test if player falls on snake/ladder and adjust accordingly
  if (position_p1 %in% start_pos) {
    position_p1 <- end_pos[which(start_pos == position_p1)]
  }

  # test if player has won
  if (position_p1 >= max_position) {
    break
  }

  # player 2 plays
  die_roll_p2 <- sample.int(n_sides_die, 1)
  position_p2 <- position_p2 + die_roll_p2

  # test if player falls on snake/ladder and adjust accordingly
  if (position_p2 %in% start_pos) {

```

```

    position_p2 <- end_pos[which(start_pos == position_p2)]
  }

  # test if player has won
  if (position_p2 >= max_position) {
    break
  }

  # add position data of players to game_data
  game_data <- rbind(game_data, c(turn, position_p1, position_p2))
}

# add descriptive column names
names(game_data) <- c("turn", "position_p1", "position_p2")

# calculate turn at which square 60 is passed by either player
for (row in 1:nrow(game_data)) {
  p1_above_60 <- min(which(game_data$position_p1 > 60))
  p2_above_60 <- min(which(game_data$position_p2 > 60))

  p1_pos_above_60 <- game_data$position_p1[p1_above_60]
  p2_pos_above_60 <- game_data$position_p2[p2_above_60]
}

# print result to screen
cat("Square 60 was passed by player 1 on turn", p1_above_60, "when they reached
square", p1_pos_above_60)

```

```
## Square 60 was passed by player 1 on turn 7 when they reached square 67
```

```
cat("Square 60 was passed by player 2 on turn", p2_above_60, "when they reached
square", p2_pos_above_60)
```

```
## Square 60 was passed by player 2 on turn 17 when they reached square 66
```

```

# plot position for each player, per turn, and indicate turn where 60 was passed
d
ggplot(data = game_data) +
  geom_line(aes(x = turn, y = position_p1, colour="player 1")) +
  geom_line(aes(x = turn, y = position_p2, colour = "player 2")) +
  annotate("segment", x=p1_above_60, y=p1_pos_above_60+15, yend=p1_pos_above_60
+2,
          arrow = arrow(), colour = "#b80b0e") +
  annotate("segment", x=p2_above_60, y=p2_pos_above_60+15, yend=p2_pos_above_60
+2,
          arrow = arrow(), colour = "#b80b0e") +
  xlab("Turn") +
  ylab("Player's position on board") +
  annotate("text", x = 5, y = 75,
          label = paste0("Player 1\nTurn: ", p1_above_60, "\nPosition: ", p1_pos_
os_above_60)) +
  annotate("text", x = 15, y = 75,
          label = paste0("Player 2 \nTurn: ", p2_above_60, "\nPosition: ", p2_
pos_above_60)) +
  scale_x_continuous(breaks = seq(1, 100, by= 2)) +
  scale_y_continuous(breaks = seq(0, 100, by = 10)) +
  theme_light() +
  theme(legend.title=element_blank(),
        legend.text=element_text(size=12))

```

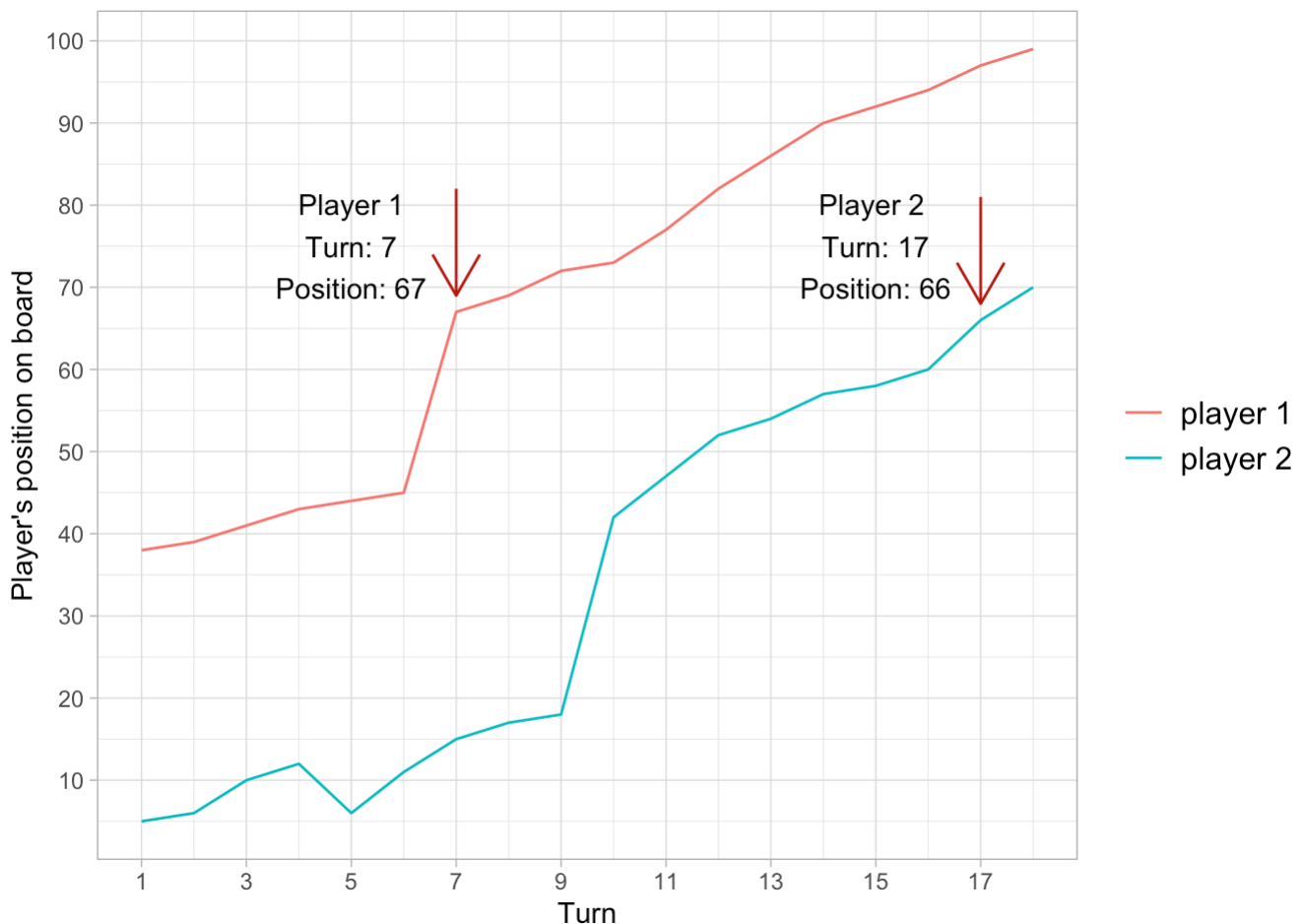


Figure 1. The relationship between turn number (x-axis) and position on the board (y-axis) in a 2-person game of snakes and ladders. The position at which each player passes position 60 is indicated by a red arrow.

Player 1 reaches (and passes) square 60 before player 2 (Fig. 1). Neither player pass 60 again, so it is not necessary to analyse for additional passes of this square in this specific game.

Question 3

- Aims:
 - Calculate number of turns to reach square 100
 - Repeat above 1000 times
 - Plot the distribution of the number of turns taken to reach the 100th square
 - Explain observations

```

# set seed for reproducibility
set.seed(42)

# same basic settings as before
max_turns <- 1000
max_position <- 100
n_sides_die <- 6

start_pos <- c(1, 4, 9, 16, 21, 28, 36, 48, 49, 51, 56, 62, 64, 71, 80, 87, 93, 95, 98)
end_pos <- c(38, 14, 31, 6, 42, 84, 44, 26, 11, 67, 53, 19, 60, 91, 100, 24, 73, 75, 78)

# play the game 1000 times and store the output in a data frame
num_iters <- 1000
iter_data <- data.frame()

# initiate the iterations
for (iter in 1:num_iters) {

  position <- 0

  # game play per iteration (1 player)
  game_data = data.frame()

  for (turn in 1:max_turns) {
    die_roll <- sample.int(n_sides_die, 1)
    position <- position + die_roll
    if (position %in% start_pos) {
      position <- end_pos[which(start_pos == position)]
    }
    if (position >= max_position) {
      break
    }
    game_data <- rbind(game_data, position)
  }

  # calculate number of turns to reach 100; add this data to data frame
  num_turns_to_100 <- nrow(game_data) + 1
  iter_data <- rbind(iter_data, num_turns_to_100)
}

# add iteration index to data frame; adjust colnames
iter_data <- cbind(iter_data, 1:nrow(iter_data))
colnames(iter_data) <- c("turns_to_100", "iteration_number")

# summary data
summary_data <- summary(iter_data$turns_to_100)

# analyse results graphically
ggplot(data = iter_data, aes(x=turns_to_100)) +

```

```
geom_histogram(colour = "#b80b0e", fill = "white") +
annotate("text", x = 170, y = 140,
  label = paste0("Median = ", summary_data[3],
    "\nMean = ", summary_data[4])) +
theme_light() +
xlab("Number of turns to reach square 100 in snakes and ladders") +
ylab("Frequency")
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

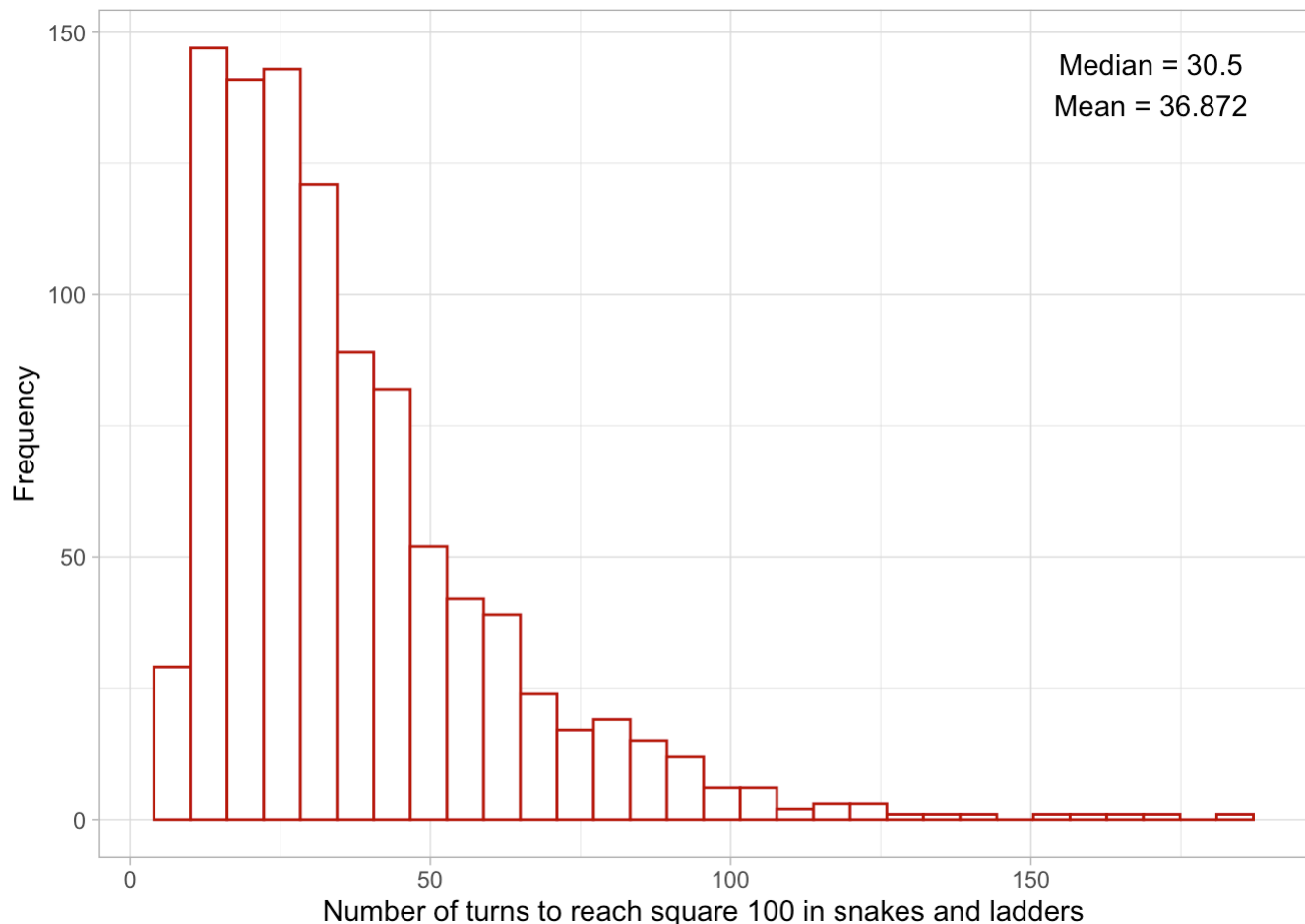


Figure 2. The frequency distribution of turns taken to win a game of snakes and ladders on a 10x10 game board, when there is a single player.

When repeating the snakes and ladders game 1000 times, the mean number of turns it takes to reach square 100 (by one player) is **36.872** (Fig. 2). The median number of turns is **30.5**. The distribution shows a clear **right skew**. Because we roll a single die (max value 6) and we need to reach square 100, the lowest number of turns to reach 100 on a board with no snakes or ladders is 17 ($6 * 17 = 102$). However, the expected value from a single roll of the die is 3.5, therefore we might expect (on average) to see that most games on a board without snakes or ladders require 29 turns to reach square 100 (because $3.5 * 29 = 101.5$). This could be considered the lower boundary on a feature-less board (no snakes or ladders).

Since the die we use is unbiased, the only other sources of variation are the number, position and length of the snakes and the ladders. The outcome in our simulated game is a function of these parameters - if they change, it could affect the shape of the observed distribution. If there were many more snakes overall, for example, a player would require more turns to finish the game compared to the current state (the mean and median would shift to the right, but the right skew would remain). Additionally, if the arrangement or length of snakes or ladders were different then we could anticipate a change in the number of turns

required to finish the game. For example, if we had 6 snakes in the final row of the board, the probability of landing on a snake would be greater (6/10 compared to current probability of 3/10) and therefore it would take longer to reach square 100. In terms of the shape of the distribution in this experiment, the lower boundary for the data (the shortest path to square 100) is 7. This is supported by the observation that the lowest number of turns in the data set is 7. The lower boundary will always be associated with the shortest path to the finish. The right-skewed tail of the distribution doesn't have this hard boundary and can, theoretically speaking, be infinite (if you are extremely unlucky in the game!).

2. Data analysis: Differential gene expression analysis

Differential gene expression analysis aims to find genes whose expression changes in response to treatment or between groups. This method compares a quantified measure of transcripts (e.g. transcript abundance) from thousands of genes between different sample types, and identifies genes that show significantly different expression levels between control and treatment groups. These can then be used as diagnostic and prognostic biomarkers (e.g. Wei et al, 2020). Gene expression studies return RNA-seq data, which is non-linear. For this reason we use non-linear distributions to model it (negative binomial or Poisson) (Rapaport et al, 2013)

In this exercise I will first comment on existing code (Question 4).

I will then do some exploratory data analysis (Question 5):

- to identify a problematic sample
- to explain why it is problematic and provide a plot to support my answer

I will then perform a regression-based differential expression analysis (Question 6).

- using the `glm` function from the `MASS` package
- to identify differentially expressed genes
- investigate the effect of removing tissue type as covariate on the p-values of the model
- identify the covariate with the biggest effect and provide supporting graphs, tables and analysis

References

Rapaport, F., Khanin, R., Liang, Y., et al. (2013). Comprehensive evaluation of differential gene expression analysis methods for RNA-seq data. *Genome biology*, 14, pp.1-13.

Wei, F. Z., Mei, S. W., Wang, Z. J., et al. (2020). Differential expression analysis revealing CLCA1 to be a prognostic and diagnostic biomarker for colorectal cancer. *Frontiers in Oncology*, 10, 573295.

Question 4

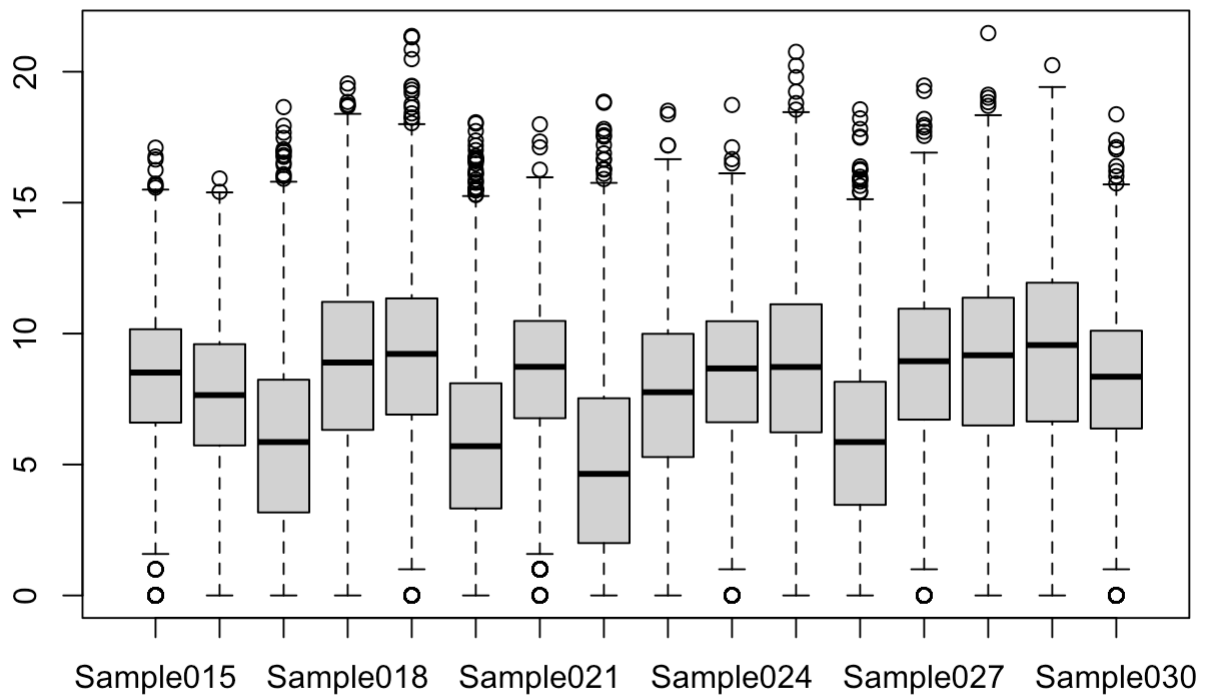
- Aim: comment on code

```
# load packages required to manipulate/tidy data (tidyverse) and for modelling (MASS)
library(tidyverse)
library(MASS)
```

```
##  
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':  
##  
##      select
```

```
# load data into coding environment  
load("data/assess_data_24-resit.Rdata")  
  
# define index value for row 20  
idx <- 20  
# define index reference for subset of samples  
c_cl <- 15:30  
  
# visualise log2 expression values for all genes (rows) but only selected samples  
boxplot(log2(Y[, c_cl] + 1))
```



```

# define x and z covariates according to pre-defined subset (cols 15:30)
x <- patient_data$tissue[c_cl]
z <- patient_data$patient[c_cl]

# subset data using indices defined above: gene in row 20 and samples in col 1
5:30
# create a new variable lib_size defined as the size of the expression library
for that sample (sum of all counts)
tmp <- data.frame(y = Y[idx, c_cl], x = x, z = z, lib_size = colSums(Y[, c_c
l]))

# apply a generalised linear model (Poisson) to the subset of data
out <- glm(y ~ x + z + lib_size, data = tmp, family = "poisson")

# extract the p-value of the x covariate (sample type) to see if it is signific
ant (and should be included in model)
p_val <- summary(out)$coefficients[2, 4]

```

Question 5

- Aims:
 - Identify problematic sample
 - Explain why problematic with plot

```

# draw diagnostic boxplot of all gene expression data to visualise min, max, me
dian and quartiles; same data as would be given in summary(Y)
boxplot(log2(Y + 1), border = "#b80b0e", col = "white", xlab = "Sample", ylab =
"Log2 count")

```

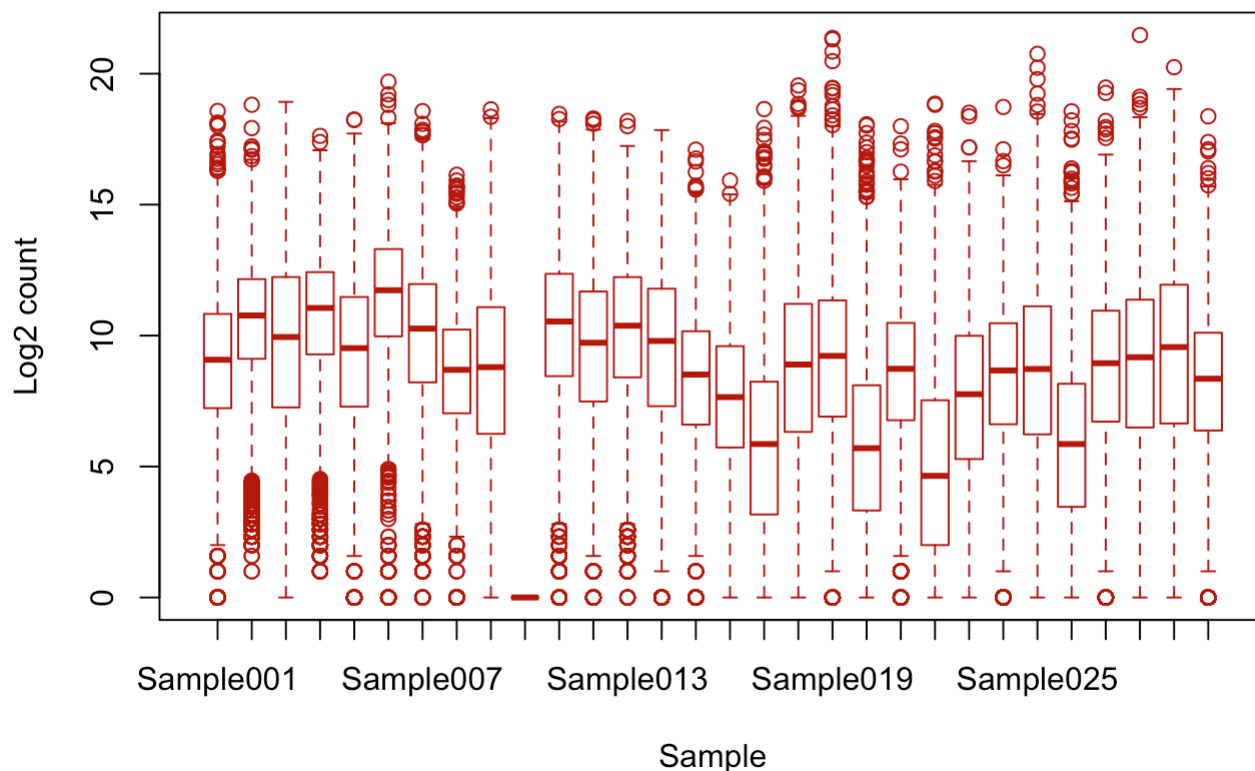


Figure 3. Diagnostic box plot showing minimum, maximum, median and quartiles of gene expression data. Outliers are shown as open circles.

One sample has no distribution at all (Fig. 3). It is difficult to identify it given the x axis, so I will plot only the means of the samples and identify it that way.

```
# confirm problematic sample with dot plot
plot(colMeans(log2(Y+1)),
     xlab = "Sample index",
     ylab = "Mean Log2 Count",
     col = "#b80b0e")
```

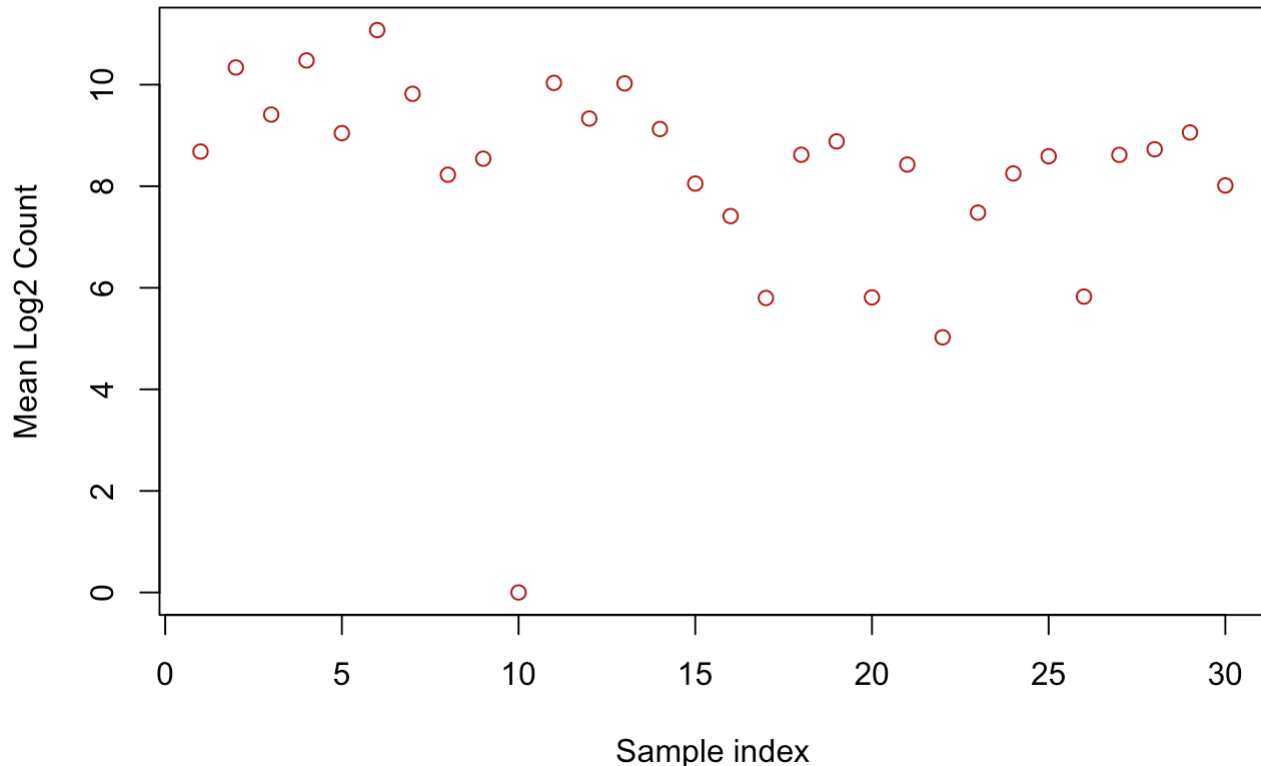


Figure 4. Dot plot showing the mean expression value for all genes per sample.

Sample 10 is problematic as it has no data (Fig. 4)! I will remove this sample from my data set before further analysis. To note: these are paired samples, so patient 10 will have two samples in the data set; it can be argued that the unmatched sample should also be removed if pairwise analysis is done. However, based on how the data will be analysed and in the interest of not losing any useful information, I will leave it in for now.

Question 6

- Aims:
 - Remove troublesome sample
 - Apply a generalised linear model to the data
 - Identify differentially expressed genes
 - Investigate the effect of removing tissue type as covariate on the p-values of the model
 - Identify the covariate with the biggest effect and provide supporting graphs, tables and analysis

```

# Remove sample 10 from patient data and gene expression data
Y <- Y[,-10]
patient_data <- patient_data |>
  filter(sample_id != "Sample010")

# define covariates
x <- patient_data$tissue
z <- patient_data$patient
lib_size <- colSums(Y)

# define number of genes (p)
p <- nrow(Y)

# create empty vector to accumulate p-values from linear model
p_vals <- rep(0, p)

# initiate loop that will produce a model for each gene
# the outcome variable is gene expression level (count) and the covariates are
# tissue type, patient and library size
for (gene in 1:p) {

  tmp <- data.frame(y = Y[gene,], x = x, z = z, lib_size = lib_size)
  out <- glm(y ~ x + z + lib_size, data = tmp, family = poisson(link="log"))
  p_vals[gene] <- coef(summary(out))[2,4]

}

# Bonferroni correction for multiple comparisons
adj_p_vals <- p.adjust(p_vals, "bonferroni")

# count number of significant p values at 0.01
sig_p_val <- sum(adj_p_vals < 0.01)

# create data frame for use in plotting result
p_val_adj_df <- data.frame(p_val = adj_p_vals, idx = 1:p)

ggplot(p_val_adj_df, aes(x = idx, y = log10(p_val))) +
  geom_point(size = 1.5, col = "#b80b0e") +
  labs(y = expression("log"["10"] ~ "adjusted p-value"),
       x = "Gene index") +
  ggtitle("Model including tissue, patient and library size") +
  theme_light()

```

Model including tissue, patient and library size

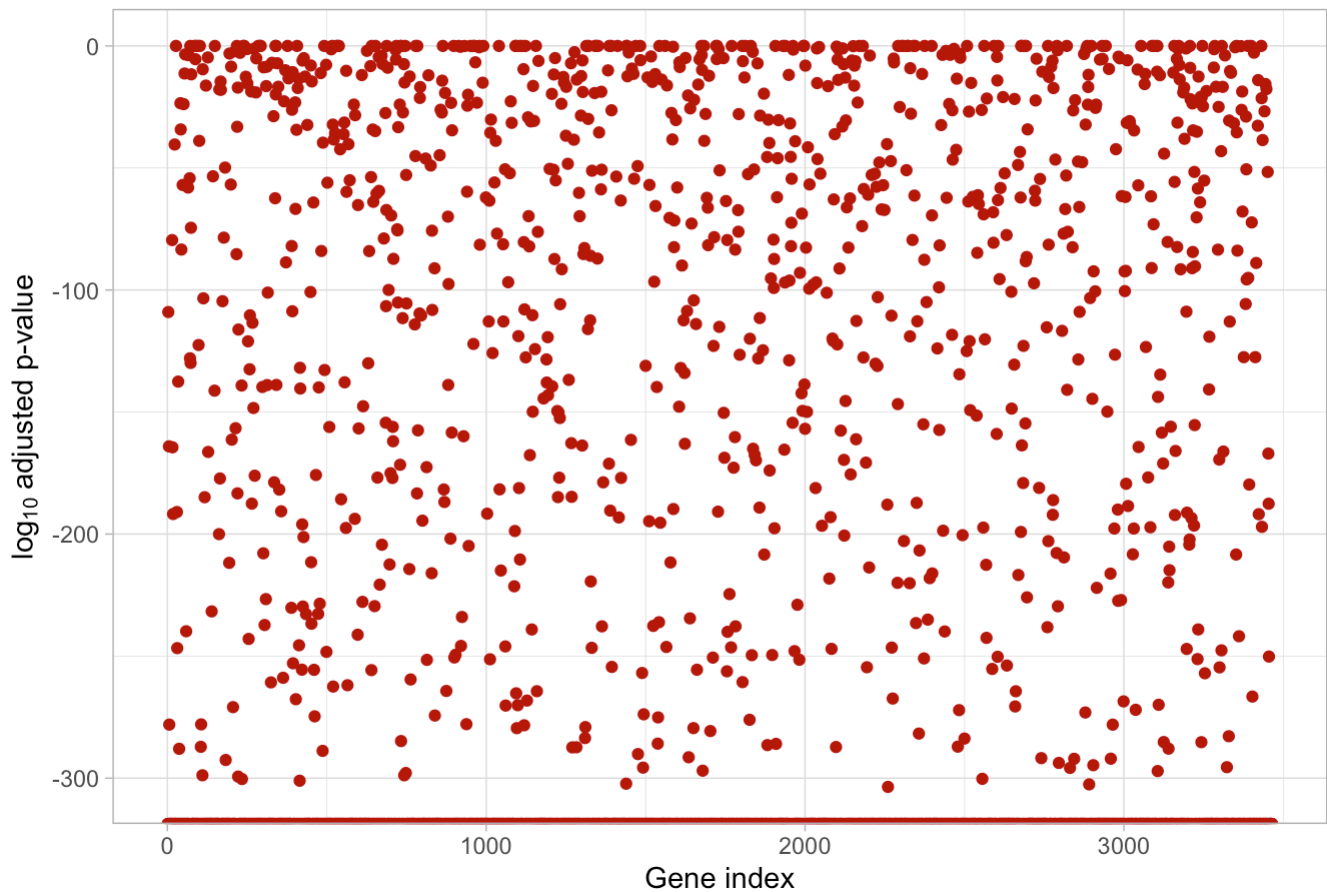


Figure 5. Significance value of the tumour covariate (Y-axis) in a generalised linear model applied to count data of 3465 genes (X-axis), where the model included tissue type (tumour/normal), patient id (patients 1-15), and library size of each sample as predictors, and expression as outcome.

The graph shows log₁₀ p-values that fall on the zero line (i.e. p-value = 1), with a dispersion downwards (Fig. 5). The result of the modelling loop shows that the coefficient for tumor type in the model is significant for 3327 out of the 3465 genes analysed at an alpha of 0.01. This seems unlikely (I assume it is difficult to find genes that so strongly relate to tumour diagnostics - finding ~ 3300 seems rare!). Furthermore, including `patient_id` as a covariate appears to be overfitting of the data since there is no way to generalise this model to others if it is specifically fit to these patients. Therefore a better model would exclude `patient_id` as covariate. Below I will re-do the modeling excluding this covariate.

```

p_vals_2var <- rep(0, p)

for (gene in 1:p) {

  tmp <- data.frame(y = Y[gene,], x = x, lib_size = lib_size)
  out <- glm(y ~ x + lib_size, data = tmp, family = poisson(link="log"))
  p_vals_2var[gene] <- coef(summary(out))[2,4]

}

# Bonferroni correction for multiple comparisons
adj_p_vals_2var <- p.adjust(p_vals_2var, "bonferroni")

# count number of significant p values at 0.01
sig_p_val2 <- sum(adj_p_vals_2var < 0.01)

# data frame for plotting
p_val_adj_2var_df <- data.frame(p_val = adj_p_vals_2var, idx = 1:p)

ggplot(p_val_adj_2var_df, aes(x = idx, y = log10(p_val))) +
  geom_point(size = 1.5, col = "#b80b0e") +
  labs(y = expression("log"["10"] ~ "adjusted p-value"),
       x = "Gene index") +
  ggtitle("Model including tissue and library size") +
  theme_light()

```

Model including tissue and library size

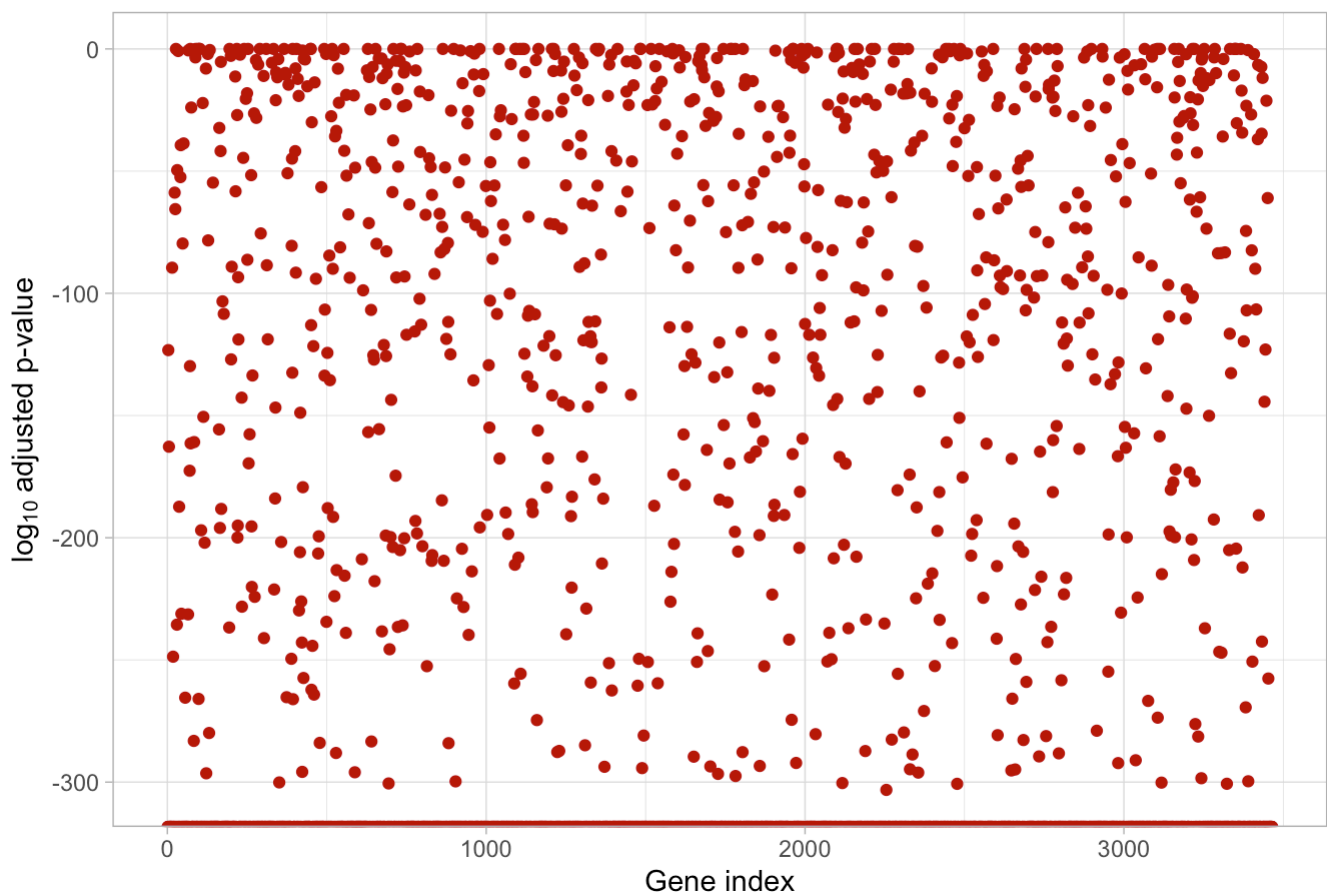


Figure 6. Significance value of the tumour covariate (Y-axis) in a generalised linear model applied to count data of 3465 genes (X-axis), where the model included tissue type (tumour/normal) and library size of each sample as predictors, and expression as outcome.

This model returns significant p-values for 3347 out of 3465 genes (20 more genes than before) (Fig. 6). The next step will be to investigate `library_size` on its own (i.e. investigating the effect of removing tissue from the model as required in **Question 6**).

```
p_vals_no_tissue <- rep(0, p)

for (gene in 1:p) {

  tmp <- data.frame(y = Y[gene,], lib_size = lib_size)
  out <- glm(y ~ lib_size, data = tmp, family = poisson(link="log"))
  p_vals_no_tissue[gene] <- coef(summary(out))[2,4]

}

# Bonferroni correction for multiple comparisons
adj_p_vals_no_tissue <- p.adjust(p_vals_no_tissue, "bonferroni")

# count number of significant p values at 0.01
sig_p_val3 <- sum(adj_p_vals_no_tissue < 0.01)

# data frame for plotting
p_val_adj_no_tissue_df <- data.frame(p_val = adj_p_vals_no_tissue, idx = 1:p)

ggplot(p_val_adj_no_tissue_df, aes(x = idx, y = log10(p_val))) +
  geom_point(size = 1.5, col = "#b80b0e") +
  labs(y = expression("log"["10"] ~ "adjusted p-value"),
       x = "Gene index") +
  ggtitle("Model including library size only") +
  theme_light()
```

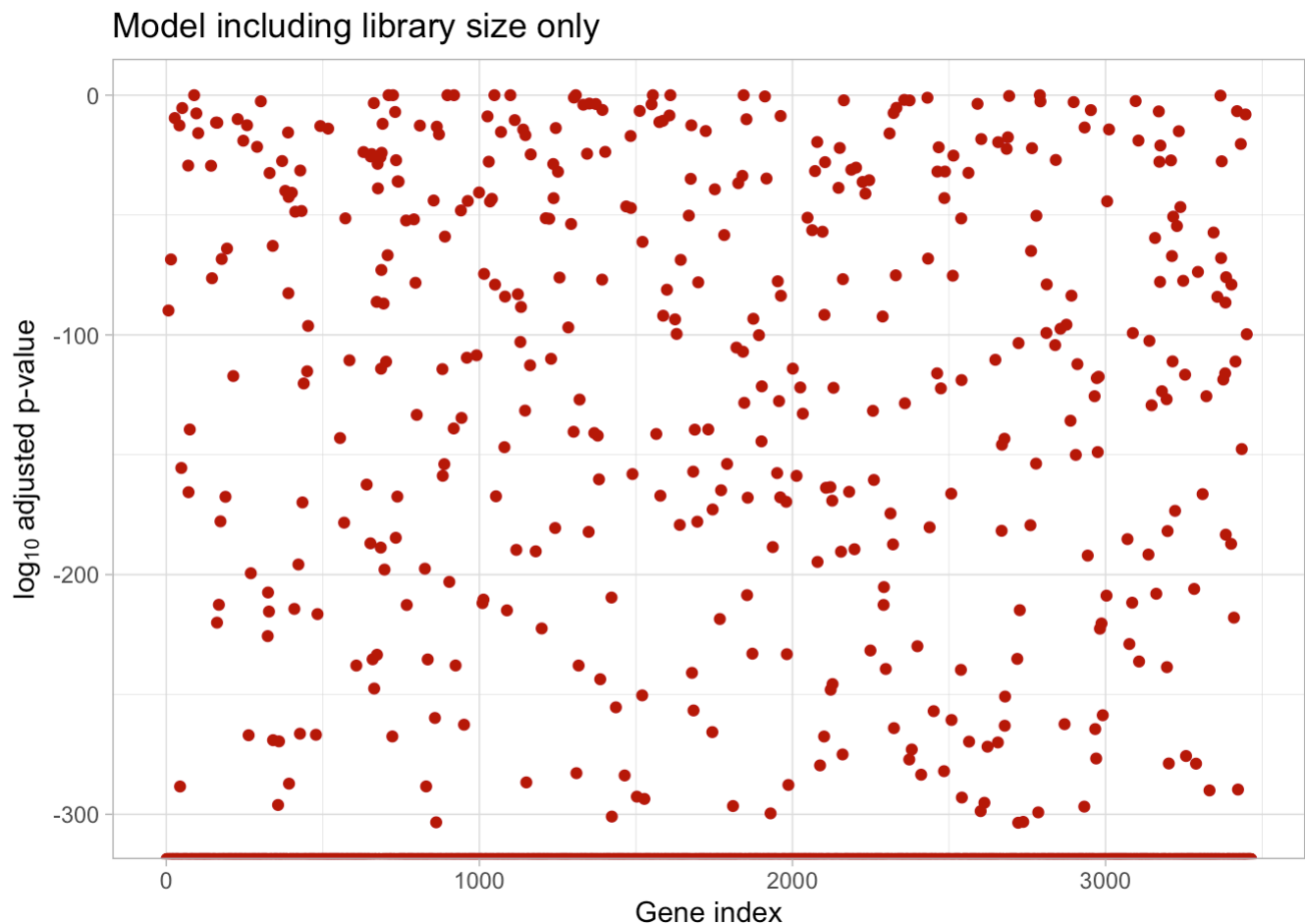


Figure 7. Significance value of the `library_size` covariate (Y-axis) in a generalised linear model applied to count data of 3465 genes (X-axis), where the model includes library size of each sample as predictor, and expression as outcome.

Here the number of log₁₀ adjusted p-values at a value of zero have reduced visibly (i.e. fewer p-values = 1) (Fig. 7). The model excluding `tissue` and only including `library_size` returns significant values for more genes than in the previous models: here we have 3447 significant p-values out of 3465 genes. This means that gene expression (count) can be significantly described simply by the size of the sequencing library in that sample. Therefore it actually appears (from this analysis) that one of two things are true: either the count data still need to be normalised, since the size of the count library affects the outcome of the model **OR** that none of the genes in this panel can be used to distinguish between tumour and normal tissue.

[As a comment, I'd like to add that I tried a form of normalising of the Y data by dividing the count of each gene by the total library size for each sample ($\text{normY} = Y[\text{gene,}]/\text{lib_size}$) and re-doing the model with `tissue` type only, but all of the p-values returned were close to 1, with the adjusted values being exactly 1, so I presume either I'm wrong or my technique was incorrect. Or, alternatively, that any significant output from this data set is a side-effect of the strong relationship between library size and expression count!]

In order to analyse which covariate has the biggest effect, I will further investigate the effect of adding covariates to a single, significant gene (I have selected this gene based on additional analysis below: ENSG00000068724).

```

# select gene of interest based on DESeq2 output from below
gene_of_interest <- Y["ENSG00000068724",]

# construct data frame for model building
effect_size_df <- data.frame(y = gene_of_interest, x = x, z = z, lib_size = lib_size)

# set up model including all covariates
mod1 <- glm(y ~ x + z + lib_size, data = effect_size_df, family = poisson(link = "log"))

# remove patient covariate
mod2 <- glm(y ~ x + lib_size, data = effect_size_df, family = poisson(link = "log"))

# remove library size covariate
mod3 <- glm(y ~ x, data = effect_size_df, family = poisson(link = "log"))

# test significance of included covariates
anova(mod1, mod2, test = "LRT")

```

```

## Analysis of Deviance Table
##
## Model 1: y ~ x + z + lib_size
## Model 2: y ~ x + lib_size
##   Resid. Df Resid. Dev  Df Deviance   Pr(>Chi)
## 1         12      15567
## 2         26      87798 -14   -72231 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

anova(mod2, mod3, test = "LRT")

```

```

## Analysis of Deviance Table
##
## Model 1: y ~ x + lib_size
## Model 2: y ~ x
##   Resid. Df Resid. Dev Df Deviance   Pr(>Chi)
## 1         26      87798
## 2         27     271260 -1  -183462 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

# look at coefficient values for each model individually
summary(mod1)

```

```
##
## Call:
## glm(formula = y ~ x + z + lib_size, family = poisson(link = "log"),
##      data = effect_size_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  6.708e+00  3.119e-02  215.05  <2e-16 ***
## xTumour      -3.858e+00  1.437e-02 -268.39  <2e-16 ***
## zPatient.10  2.507e+00  3.546e-02   70.69  <2e-16 ***
## zPatient.11  1.980e+00  3.293e-02   60.14  <2e-16 ***
## zPatient.12  2.086e+00  3.312e-02   63.00  <2e-16 ***
## zPatient.13  2.875e+00  3.524e-02   81.57  <2e-16 ***
## zPatient.14  2.862e+00  3.823e-02   74.87  <2e-16 ***
## zPatient.15 -9.260e-01  5.628e-02  -16.45  <2e-16 ***
## zPatient.2   1.640e+00  3.371e-02   48.65  <2e-16 ***
## zPatient.3   2.685e+00  3.448e-02   77.88  <2e-16 ***
## zPatient.4   2.366e+00  4.280e-02   55.28  <2e-16 ***
## zPatient.5   1.332e+00  3.459e-02   38.50  <2e-16 ***
## zPatient.6   1.511e+00  3.395e-02   44.51  <2e-16 ***
## zPatient.7   2.874e+00  3.177e-02   90.48  <2e-16 ***
## zPatient.8   2.747e+00  3.190e-02   86.12  <2e-16 ***
## zPatient.9   2.102e+00  3.264e-02   64.40  <2e-16 ***
## lib_size     5.393e-08  1.006e-09   53.60  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 608285  on 28  degrees of freedom
## Residual deviance:  15567  on 12  degrees of freedom
## AIC: 15860
##
## Number of Fisher Scoring iterations: 8
```

```
summary(mod2)
```

```
##
## Call:
## glm(formula = y ~ x + lib_size, family = poisson(link = "log"),
##      data = effect_size_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  8.739e+00  3.811e-03  2293.2   <2e-16 ***
## xTumour      -4.121e+00  1.215e-02  -339.3   <2e-16 ***
## lib_size      7.465e-08  1.709e-10   436.9   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 608285  on 28  degrees of freedom
## Residual deviance:  87798  on 26  degrees of freedom
## AIC: 88063
##
## Number of Fisher Scoring iterations: 5
```

```
summary(mod3)
```

```
##
## Call:
## glm(formula = y ~ x, family = poisson(link = "log"), data = effect_size_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  9.897895    0.001831  5406.2   <2e-16 ***
## xTumour      -3.687582    0.012117  -304.3   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 608285  on 28  degrees of freedom
## Residual deviance: 271260  on 27  degrees of freedom
## AIC: 271523
##
## Number of Fisher Scoring iterations: 5
```

```
# create data frame for AIC function application
models <- list(mod1, mod2, mod3)
models_names <- c("all", "tissue_libsize", "tissue")

# Q3.
lapply(models, AIC)
```

```
## [[1]]  
## [1] 15860.3  
##  
## [[2]]  
## [1] 88062.93  
##  
## [[3]]  
## [1] 271523.4
```

In order to identify which covariate has the biggest effect, I have focused in on a single gene for simplicity ("ENSG00000068724"). I have set up three different models to analyse the changes in coefficient size for comparison. I have done this because the significance of all covariates have been identical at $<2e-16$.

Doing an ANOVA indicates that removal of any covariates cause a significant difference in the model.

- In model 1, which includes tissue type, library size and patient as predictor variables, the variable with the largest coefficient is tissue type, where the count value decreases by 3.858 in tumour tissue compared to normal tissue (where the value is 6.708 - the intercept).
- In model 2, which includes tissue type and library size, the variable with the largest coefficient is, again, tissue type where the count value decreases by 4.121 in tumour tissue compare to normal tissue (where the value is 8.739 - the intercept).
- In model 3, which includes only tissue type, the coefficient for tumour is -3.688 compared to the intercept (normal tissue) which is 9.898.

Finally, I have used the Akaike Information Criterion to compare the 3 models. The model with the best fit is mod1 , which includes all 3 covariates. But as stated above, this increased accuracy could be a side-effect of over-fitting and a possible confounding effect of library size.

Additional analysis

Since my code above didn't give me an answer that I trust, I have tried other approaches to analyse the data. First, I have applied a simple heatmap to the data to see if there is any visible grouping and if my results above make sense. Secondly, after searching online I found that the R package DESeq2 is very popular for differential gene expression analysis. I have attempted to improve on my results by applying this. It will, however, mean that I will have to make some assumptions about the data that has been supplied to us in terms of level of processing/normalisation.

```
heatmap(Y)
```

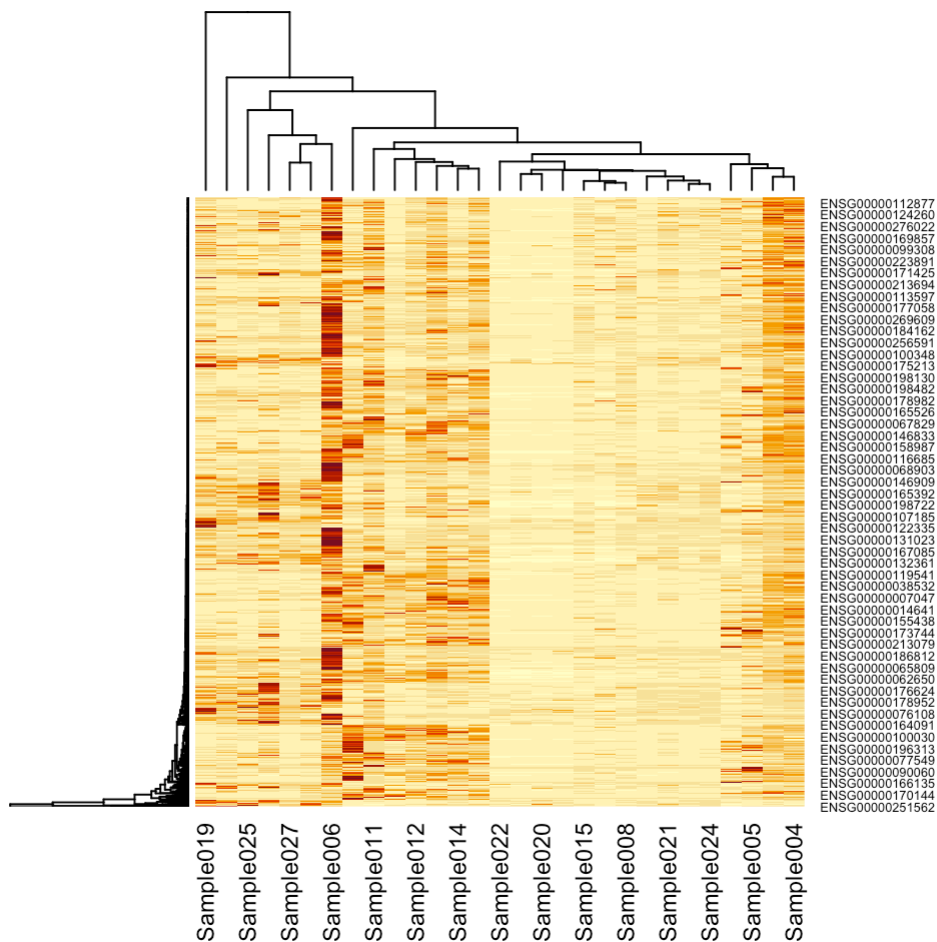


Figure 8. A heatmap of gene expression values between tumour and normal tissues.

A very basic heatmap of the original gene data shows that there are several potential clusters in the samples (Fig. 8). Visually, one could split the samples into several groups based on shading, but within these groups it is interesting to note that samples from tumours (sample001 - sample015) are distributed among samples from normal tissue (sample016 - sample030). Therefore this type of clustering does not clearly distinguish between tissue types.

Below I will analyse the data using DESeq2 and two models: one including both tissue type and library size, and one that only includes tissue type. DESeq2 uses the negative binomial distribution. Using this method, I will identify the two most significant genes identified in either model.

```
# rename data to produce second set for analysis; note: sample010 has been removed already
data <- Y
meta <- patient_data

#prepare metadata
rownames(meta) <- meta$sample_id
meta$lib_size <- colSums(data)
meta <- meta |>
  dplyr::select(-sample_id)
meta$tissue <- as.factor(meta$tissue)

### Check that sample names match in both files
all(colnames(data) %in% rownames(meta))
```

```
## [1] TRUE
```

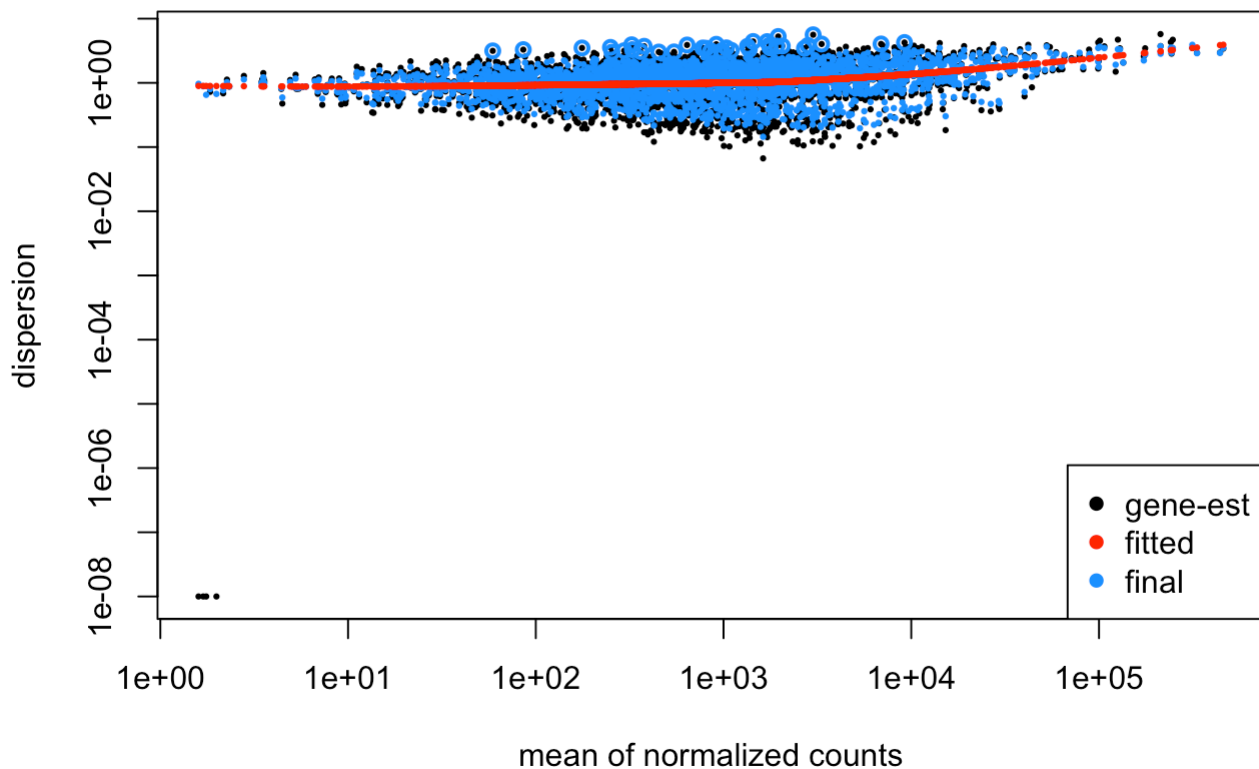
```
# install required package
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

BiocManager::install("DESeq2")
library(DESeq2)

## Create DESeq2Dataset object
dds <- DESeqDataSetFromMatrix(countData = data, colData = meta, design = ~ tissue)

## Run analysis
dds <- DESeq(dds)

# examine fit of model to data
plotDispEsts(dds)
```




```
## Define contrasts, extract results table
contrast_oe <- c("tissue", "Tumour", "Normal")

res_tableOE_unshrunk <- results(dds, contrast=contrast_oe, alpha = 0.05)
#BiocManager::install("apeglm")
#res_tableOE <- lfcShrink(dds, contrast=contrast_oe, res=res_tableOE_unshrunk)

# The order of the names determines the direction of fold change that is reported. The name provided in the second element is the level that is used as baseline. So for example, if we observe a log2 fold change of -2 this would mean the gene expression is lower in Tumour relative to Normal.

## Summarize results; notice exclusion of non-zero values in output message
summary(res_tableOE_unshrunk)
```

```
##
## out of 3465 with nonzero total read count
## adjusted p-value < 0.05
## LFC > 0 (up)      : 942, 27%
## LFC < 0 (down)    : 1032, 30%
## outliers [1]      : 0, 0%
## low counts [2]    : 0, 0%
## (mean count < 2)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```
# create data frame for filtering
output <- data.frame(res_tableOE_unshrunk)
signif_genes_tissue_mod <- output[output$padj < 0.05,]

most_sig_gene <- min(signif_genes_tissue_mod$padj)
top_gene_tissue_mod <- rownames(signif_genes_tissue_mod[signif_genes_tissue_mod$padj == most_sig_gene,])

# Plot expression for single gene
plotCounts(dds, gene=top_gene_tissue_mod, intgroup="tissue")
```

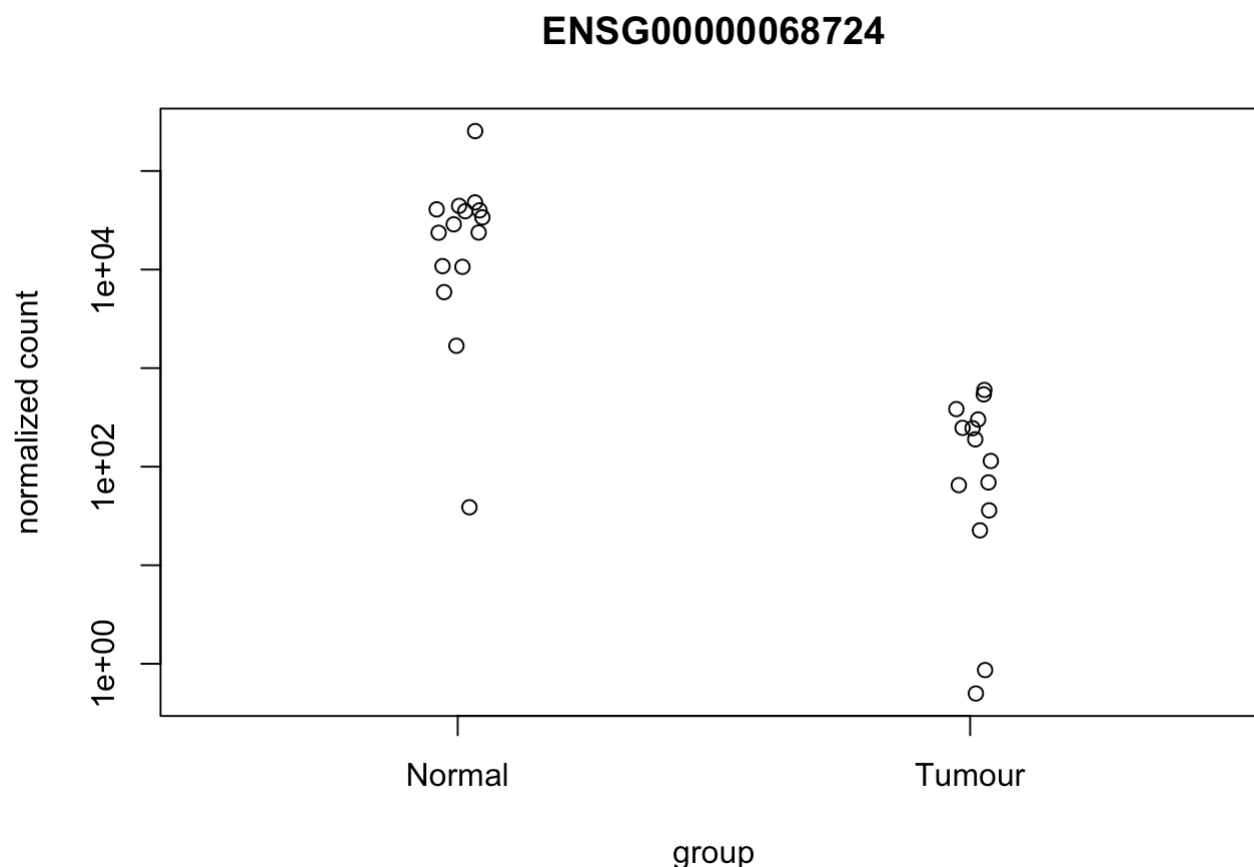


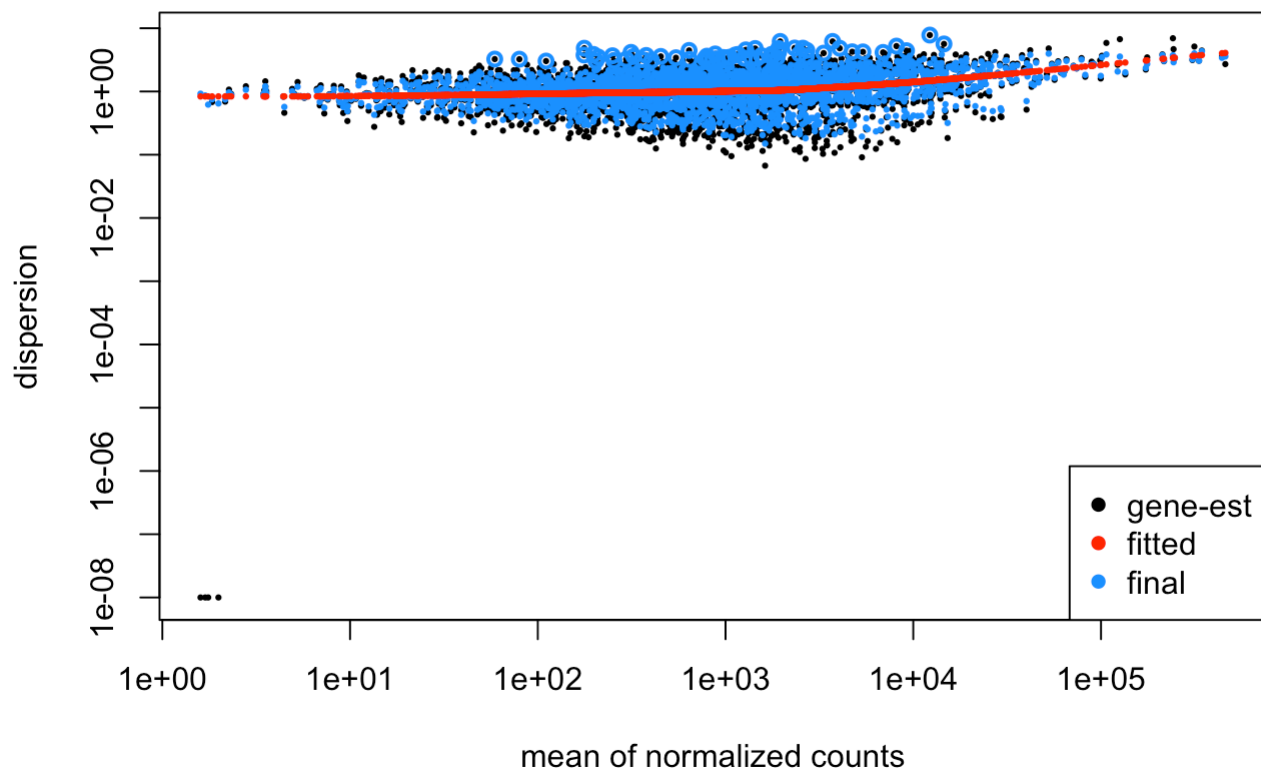
Figure 9. Normalised counts of gene ENSG00000068724 compared between normal and tumour tissue.

Using the DESeq2 package returns 1974 genes whose expression can be predicted based solely on tissue type. Plotting expression values for the gene with the lowest p-value (ENSG00000068724) seems to confirm that it is differentially expressed between normal and tumour tissue, with the tumour tissue having a lower expression level (Fig. 9).

I will repeat the analysis including library size as covariate in the model for completeness.

```
#### incorporating more than one covar
dds2 <- DESeqDataSetFromMatrix(countData = data, colData = meta, design = ~ tissue + lib_size)
## Run analysis
dds2 <- DESeq(dds2)

# examine fit of model to data
plotDispEsts(dds2)
```



```
## Define contrasts, extract results table
contrast_oe2 <- c("tissue", "Tumour", "Normal")

res_table0E_unshrunk2 <- results(dds2, contrast=contrast_oe2, alpha = 0.05)
#BiocManager::install("apeglm")

output2 <- data.frame(res_table0E_unshrunk2)
sig2 <- output2[output2$padj < 0.05,]

most_sig_gene2 <- min(sig2$padj)
top_gene2 <- rownames(sig2[sig2$padj == most_sig_gene2,])

# Plot expression for single gene
plotCounts(dds2, gene=top_gene2, intgroup="tissue")
```

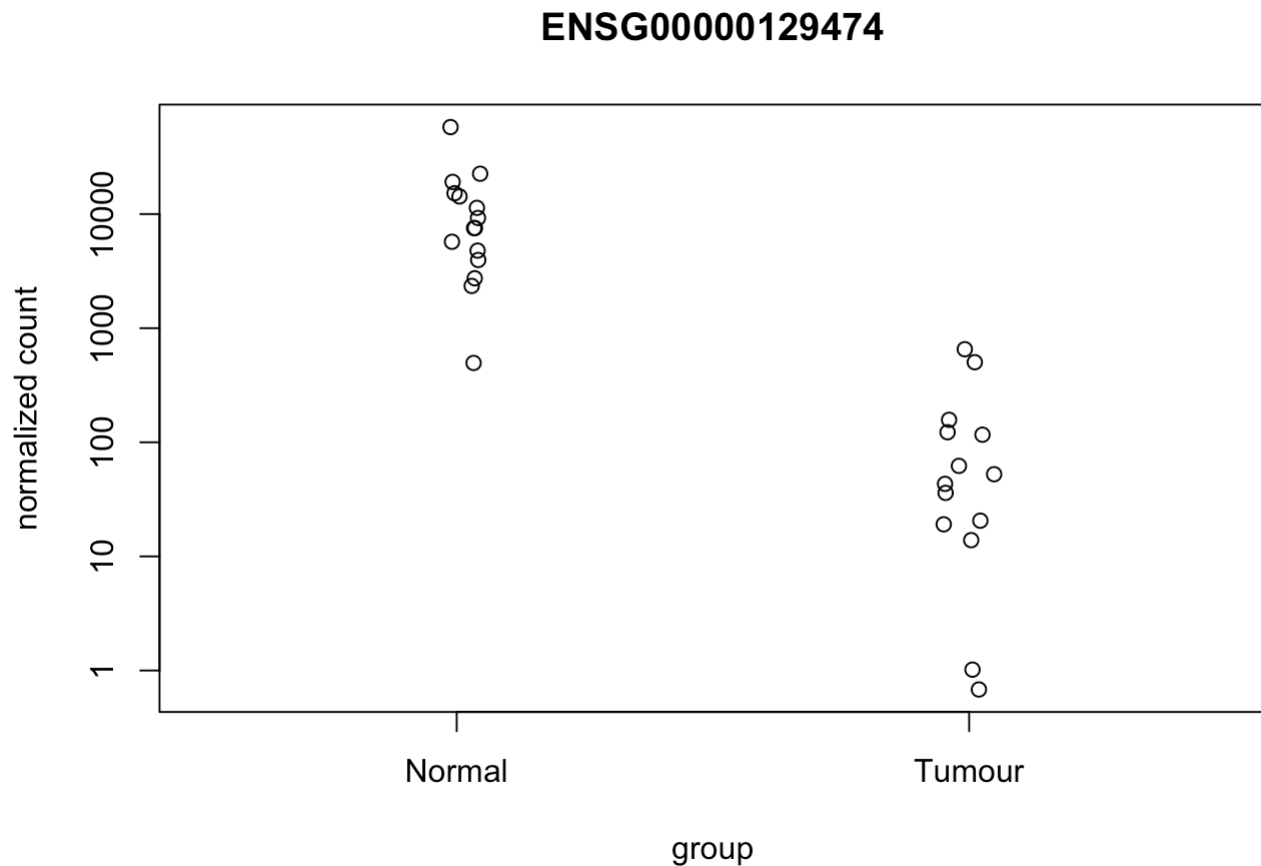


Figure 10. Normalised counts of gene ENS00000129474 compared between normal and tumour tissue.

Including library size as a covariate has decreased the number of significant genes to 1889. Plotting the gene with the highest p-value (ENSG00000129474) also confirms that this gene seems to be differentially expressed between normal and tumour tissue, being down-regulated in tumour tissue (Fig. 10). It is important to note that the package has automatically returned a warning that the design formula contains one or more numeric variable that has a mean or SD larger than 5 (an arbitrary threshold) which can induce collinearity with the intercept (data should be centered and scaled to avoid this). Since this message wasn't triggered in the previous model it can be assumed that the library size covariate introduces potential errors in model fitting.